



Figure 15-17 *After selecting Ka(y) to visit in finding the shortest path to Naur*

The algorithm selects Gr(ay) for the next visit as shown in [Figure 15-18](#). No new vertices have been added to the `costs` structure, but one has been changed. The total time taken to reach Na(ur) has been updated from 87 to 76. That's due to the new information found when visiting Ka(y) that Na(ur) could be reached in 36 minutes. Unlike a paper notebook, where you might put a line through an old entry, the previous estimate is simply erased from the entry for Na(ur), and the new entry showing the path from Kay replaces it.



Figure 15-18 *Visiting Gr(ay) in finding the shortest path to Naur*

The final shortest path to Naur is updated at the next stage, changing the total time to 74 with Gr(ay) as the parent vertex. With Naur in the `visited` array, we're done updating the `costs` structure. What's left is following the "breadcrumbs" stored in that structure to determine the best path from Blum.

We'll see how to do that and figure out what we should use to represent the `costs` notebook and how to deal with an end vertex in a different connected component in the next section.

Implementing the Algorithm

What kind of data structure should we use for the notebook? Dijkstra's algorithm needs something that records total weights (times) and predecessors for individual vertices of the graph (cities). What options do we have?

Among the data structures you've seen so far, you could use nearly all of them to store a record holding a path weight and vertex index for the predecessor. These records would be identified by an integer key, the vertex index for the destination. Using an array would give you the fastest access by that key, $O(1)$. Sorted lists, sorted arrays, trees, and heaps could be an option because that allows for quickly finding the lowest weight path. Using those structures, however, means finding the records to be updated for each newly discovered edge weight will take longer than $O(1)$; probably $O(\log N)$ or $O(N)$, where N is the number of vertices you're tracking. Like the analogy with the agents sent to the train stations, you must update several records for each vertex visited (all of its adjacent vertices minus the parent vertex), so you really need to keep the time to find each record as $O(1)$.

If you use an array, you need an array that holds all the vertices. That's not much memory for small graphs but becomes more significant with huge ones. Finding the shortest path in Turala only needed an array that could hold six items. If you were finding the shortest route between two road intersections in North America, however, there are more than a million vertices to consider. Do you really want to create an array for millions of vertices when the final result route needs to visit only a couple of dozen?

If the purpose of applying Dijkstra's algorithm is to find the shortest path from a starting vertex to *all other vertices* in the connected component, then it makes sense to allocate an array for all the vertices. They will all be visited as part of the process. If the purpose is to find the shortest path between *two specific vertices*, however, it makes more sense to use a hash table. After you visit the destination vertex, the algorithm can stop and return the shortest (lowest weight) path. There will be entries for all the vertices that are adjacent to a vertex on the shortest path, but that could be millions fewer.

As you saw in [Chapter 11](#), hash tables can grow as needed while still providing $O(1)$ search time. Using Dijkstra's algorithm to search for the shortest path between two out of a two million vertex road network is likely to need to explore a tiny fraction of the total number of vertices, maybe tens or hundreds of vertices. A hash table allows you not only to use memory proportional to that tiny fraction of vertices, but it also means that enumerating (traversing) all the current vertices in the table to, say, find the one with lowest total path weight takes time proportional to the tiny fraction. That's not as efficient as keeping all the items in a priority queue or heap where the time to find the shortest takes $O(1)$, but those structures would take more time to find each vertex when updating their total path cost.

You should also consider how to track vertices that have been visited by the algorithm. In the notebook, you crossed off cities (vertices) as they were visited. That suggests that you could add a flag to each record indicating whether the vertex had been visited or not. With that representation, determining whether vertex *K* has been visited or not takes a hash table search, and if a record for *K* is found, checking if its flag is set. That is probably the most memory-efficient way, but you could also keep a separate hash table to store the visited vertices. Doing so means determining the visit status for *K* only requires seeing whether key *K* has been inserted in the hash table.

In the Visualization tool, the costs notebook and visited table appear as simple tables or arrays. That is only to simplify their appearance in the tool; we use hash tables in the code.

Python Code

You can implement the shortest-path algorithm in a single method as shown in [Listing 15-3](#). This version will finish as soon as it finds the shortest path between the given `start` and `end` vertices, not continuing to find the shortest paths to all other vertices.

Listing 15-3 *The `shortestPath()` Method for `WeightedGraph`*

```
class WeightedGraph(object): # A graph containing vertices and edges
...

    def shortestPath(          # Find shortest path between two vertices,
        self, start, end):    # if it exists, as list of vertex indices
```

```

visited = {}          # Hash table of visited vertices
costs = {              # Hash of path costs to vertices including
    start: (0, start)} # their predecessor vertex
while end not in visited: # Loop until we visit the end vertex
    nextVert, cost = None, math.inf # Look for next vertex
    for vertex in costs: # among unvisited vertices whose cost
        if (vertex not in visited and # to reach is the lowest
            costs[vertex][0] <= cost):
            nextVert, cost = vertex, costs[vertex][0]
    if nextVert is None: # If no unvisited vertex could be found
        break           # we cannot get to the end, so exit loop
    visited[nextVert] = 1 # Visit vertex at end of lowest cost
    for adj in self.adjacentVertices(nextVert): # path and
        if adj not in visited: # adjacent, unvisited vertices
            pathCost = (      # Extended path costs weight of adj.
                self.edgeWeight(nextVert, adj) + # edge plus cost of
                costs[nextVert][0]) # path so far
            if (adj not in costs or # If reached adj for first time
                costs[adj][0] > pathCost): # or old path costlier,
                costs[adj] = ( # update cost to reach vertex
                    pathCost, nextVert) # via extended path

path = []              # Build output path from end
while end in visited: # Path only contains visited vertices
    path.append(end)   # Append end vertex
    if end == start:   # If we reached the start,
        break         # we're done. Otherwise go to
    end = costs[end][1] # predecessor via lowest cost path
return list(reversed(path)) # Return path from start to end

```

As explained in the previous section, the `shortestPath()` method starts off by creating two hash tables: `visited` and `costs`. The Python dict structures used for these hash tables simplify the syntax a little, but you could also use instances of the `HashTable` class introduced in [Chapter 11](#). The `visited` table is initially empty because no vertices have yet been visited. As the algorithm visits vertices, it will insert a 1 in this hash table at the vertex's index.

The `costs` hash table maps vertex indices to the records containing the total path weight to that vertex and the predecessor vertex index to reach it. This holds the notebook contents and is the same as what is shown on the right side of [Figures 15-10, 15-11, 15-12, and 15-13](#). Initially, it holds a record of `(0, start)`, as shown in [Figure 15-10](#). That represents the zero path weight of the zero length path from the starting vertex. The first element of the record (tuple) is the path length, and the second element is the predecessor or parent vertex.

For the starting vertex, there is no predecessor, so we could store `None` in the tuple instead of `start`. The figure shows the record being placed in the first cell of the hash table, but the starting index could hash to any cell.

With these two hash tables set up, we now enter the main `while` loop of the algorithm. The loop condition, `end not in visited`, says it all. The loop continues until the `end` vertex is inserted in the visited hash table. (If the `start` and `end` vertices are not in the same connected component, we'll detect that later and break out of the loop.)

In the loop body, the first thing to do is find which vertex to visit next. The rule in Dijkstra's algorithm is to find the unvisited vertex with the lowest weight path from the start. We set up a `nextVert` and a `cost` variable to search for these. The `cost` variable is initialized to `math.inf` so that any (finite) cost found is considered as a lower-cost path. The first inner `for` loop goes over all the vertices with records in the `costs` hash table. Inside that `for` loop, when it finds a `vertex` that has not yet been visited and the path cost to reach that vertex is less than the minimum cost path found so far, it updates `nextVert` and `cost` to use that `vertex` and its path cost (stored in the first element `[0]` of the `costs` record).

We now know which vertex to visit. Because we put the `start` vertex in the `costs` table outside of the main loop, it will be found by the inner loop and become the first vertex visited. If the inner `for` loop ended without finding an unvisited vertex, that means that we've explored all the vertices connected to the starting vertex without discovering a path to the destination. In other words, if the `nextVert` is `None`, then we can break out of the main loop.

When the `nextVert` is not `None`, we mark it in the `visited` hash table and proceed to process the edges connected to it. The second inner `for` loop steps through all adjacent vertices of `nextVert`, storing the index in the `adj` variable. Adjacent vertices that have not been visited are examined. The cost of the edge from `nextVert` to the `adj` vertex is added to the path cost of reaching `nextVert`. For the starting vertex, the path cost was set to zero, so no special handling is needed for the first edges. The next `if` statement checks whether `adj` is either a new vertex in the `costs` hash table or the `pathCost` to reach it is lower than the one recorded in `costs`. If it's new or lower, the `costs` hash table entry for the `adj` vertex is updated with the new lower cost and its predecessor vertex.

That takes care of the one rule and update strategy in Dijkstra's algorithm. When the main `while` loop ends, we'll either have the `end` vertex among those

visited along with its path cost in the `costs` table, or no path could be found. We assume the latter, setting `path` to `[]`, an empty list. The next loop reconstructs the path from the `costs` table, if there is one.

The final `while` loop starts from the `end` vertex and works backward. It appends the `end` vertex to the output `list`. Next, it checks whether that `end` vertex was the `start` vertex. If so, the path is complete, and it breaks out of the loop. If not, it sets `end` to the predecessor vertex index that was stored as the second element of the `costs` table record and continues on. When the loop exits, `path` holds the vertex indices of the shortest path but in reverse order. We use the Python utilities `reversed()` and `list()` to put the list in forward order and return it as a list.

This implementation is straightforward and fairly efficient. To see how all the different data structures work together to achieve the goal, it helps to run through a few examples, either by hand or using the Visualization tool.

The All-Pairs Shortest-Path Problem

In discussing connectivity in [Chapter 14](#), we wanted to know whether it was possible to travel from Athens to Kamchatka if we didn't care how many stops were made. With weighted graphs you can answer other questions that might occur to you as you wait at the ticket counter: How much will the journey cost? How long will the journey take?

To find whether a trip was possible, you created a connectivity matrix starting from the adjacency matrix. The analogous table for weighted graphs gives the minimum cost from any vertex to any other vertex using multiple edges. This is called the **all-pairs shortest-path** problem.

You might create such a matrix by

1. Making a `shortestPaths()` method based on the `shortestPath()` method that takes a single vertex as its parameter,
2. Changing its main `while` loop to end when all vertices have been visited, and
3. Returning paths and their costs for all visited vertices.

Then you could run `shortestPaths()` starting at each vertex in turn to populate the rows of the all-pairs shortest-paths matrix. With a nondirectional graph, the

result matrix would be symmetric, and you could limit the amount of work done by only determining the cost of paths where the start index is less than the end index, but it would still be quite complex.

In the end, this could produce a travel time table for Turala like the one in [Table 15-6](#), if you assume the return travel time is the same as the forward travel times shown in [Figure 15-13](#). The best time found for the trip from Blum to Naur, 74 minutes, is in the table, but it also would find that the Cerf to Naur trip could be done in 62 minutes compared to the 65-minute direct route.

Table 15-6 *All-Pairs Shortest-Path Table*

	Blum	Cerf	Dahl	Gray	Kay	Naur
Blum	—	22	16	44	40	74
Cerf	22	—	29	34	26	62
Dahl	16	29	—	28	24	58
Gray	44	34	28	—	25	30
Kay	40	26	24	25	—	36
Naur	74	62	58	30	36	—

In the preceding chapter, you saw that Warshall’s algorithm was a way to create a table showing which vertices could be reached from a given vertex using one or many steps. Analogous approaches for weighted graphs were published by Robert Floyd and Stephen Warshall separately in 1962. The pattern of moving through the adjacency matrix is the same, but the update strategy is different because you need to sum the edge/path weights rather than doing inclusive OR operations.

Let’s learn the Floyd-Warshall algorithm using a simple four-vertex graph. [Figure 15-19](#) shows a weighted, directed graph and its weighted adjacency matrix in two formats.



Figure 15-19 *A weighted, directed graph and its adjacency matrix*

The weighted adjacency matrix shows the cost of all the one-edge paths. The format on the right explicitly fills in an edge weight of positive infinity for nonadjacent vertices. The goal is to extend this matrix to show the cost of all paths regardless of length. Any path that doesn't exist will have infinite weight.

For humans, it's easy to see that you can go from B to C at a cost of 30 (10 from B to D plus 20 from D to C). The return trip, from C to B, is not possible (or at least would cost more than anyone with finite resources can afford).

As in Warshall's algorithm, you copy the adjacency matrix and then systematically modify it into a path weight matrix. You examine every cell in every row. If there's a finite weight (let's skip down to row C where there's a 30 in column A), then you then look in column C (because C is the row where the 30 is). If you find a finite entry in column C (like the 20 at row D), you know there is a path from C to A with a weight of 30 and a path from D to C with a weight of 20. From this, you can deduce that there's a two-edge path from D to A with a weight of 50. [Figure 15-20](#) shows the steps when the Floyd-Warshall algorithm is applied to the graph in [Figure 15-19](#).



Figure 15-20 *Steps in the Floyd-Warshall algorithm to create the path weight matrix*

Row A has no finite weights, so there's nothing to do there. In row B there's a 70 in column A and a 10 in column D, but there are no finite weights in column B, so the entries in row B can't be combined with any edges ending on B. (Technically, you could add the 70 to infinity, but that gives infinity and you only make changes if it lowers a path weight.)

In row C, however, you find a 30 at column A. Looking in column C, you find a 20 at row D. Now you have C to A with a weight of 30 and D to C with a weight of 20, so you have D to A with a weight of 50.

Row D shows an interesting situation: where the algorithm updates an existing cost. There's a 50 in column A. There's also 10 in row B of column D, so you know there's a path from B to A with a cost of 60. There's already a cost of 70 in this cell. What do you do? Because you want the *shortest* path and 60 is less than 70, you replace the 70 with 60.

The implementation of the Floyd-Warshall algorithm is similar to that for Warshall's algorithm. Instead of simply inserting 1s into the table when a multiple-edge path is found, you add the costs of the two paths and insert the sum, as long as the sum is less than the cost already stored for the combined path. Similar to the Dijkstra algorithm, each cell contains the minimum path weight connecting two vertices that have been found so far. After going through all the rows and columns, the overall shortest-path weights are discovered. We leave the details as an exercise.

Efficiency

So far, we haven't discussed much about the efficiency of the various graph algorithms. The issue is complicated by the two ways of representing graphs: the adjacency matrix and adjacency lists, and the fact that there are two different measures of graph size.

If an adjacency matrix is used, the algorithms we've discussed mostly require $O(V^2)$ time, where V is the number of vertices. Why? If you analyze the algorithms, you'll see that they involve examining each vertex once, and for that vertex going across its row in the adjacency matrix, looking at each edge in turn. In other words, each cell of the adjacency matrix, which has V^2 cells, is examined.

For large matrices $O(V^2)$ isn't very good performance. If the graph is dense, there isn't much you can do about improving this performance. As we noted earlier, by *dense*, we mean a graph that has many edges—one in which many or most of the cells in the adjacency matrix are filled. It's worthwhile noting that the maximum density graphs, ones where every vertex is adjacent to every other vertex, have $V \times (V - 1) / 2$ or $O(V^2)$ edges.

Many graphs are *sparse*, the opposite of dense. There's no clear-cut definition of how many edges a graph must have to be described as sparse or dense, but if each vertex in a large graph is connected by only a few edges, the graph would normally be described as sparse.

Consider again finding the shortest route between two road intersections. There could be millions of intersections for a graph covering a large continent. The intersections are the vertices of the graph, and the roads connecting them are the edges. Maybe the busiest intersection has 12 roads connected to it (such as the l'Arc de Triomphe de l'Etoile in Paris, France). Twelve is much smaller than a million, and even if every vertex had 12 edges, a million-vertex graph would need to look at most 12 million edges. That's vastly smaller than what a dense graph would have: $O(V^2)$ or millions squared, which are trillions. All but the smallest road network graphs are sparse.

In sparse graphs, running times can be improved by using the adjacency list representation rather than the adjacency matrix. This is easy to understand: you don't waste time examining adjacency matrix cells that don't hold edges. It also takes less memory because you don't need an array cell for every vertex pair—only enough to hold each edge. That's an advantage of the hash table representation of the adjacency matrix too.

For unweighted graphs, a depth-first search with adjacency lists requires $O(V+E)$ time, where V is the number of vertices and E is the number of edges. The traversal visits each vertex and each vertex's edges exactly once. For weighted graphs, both the minimum spanning tree and the shortest-path algorithm require $O((E+V) \times \log V)$ time. They, too, must visit every vertex and edge and choose the lowest-cost path at each iteration. The lowest-cost edge is kept in a heap with $O(\log N)$ removal time for the MST. (For the shortest-path algorithm, we would need to introduce another data structure to get $O(\log N)$ time to find the shortest cumulative path so far.) In large, sparse graphs these $O((E+V) \times \log V)$ times can represent dramatic improvements over the $O(V^2)$ adjacency matrix approach.

There's another kind of complexity—one that's especially important for new programmers. That's the intellectual complexity of the code. We've used the adjacency matrix approach throughout this chapter to make the code easier to read and the algorithmic steps easier to visualize. You can consult Sedgewick (see [Appendix B](#), “[Further Reading](#)”) and other writers for examples of graph algorithms using the adjacency-list approach. The adjacency list isn't much more intellectually complex and definitely outperforms the matrix on sparse graphs. The hash tables we've used for some of the structures like the costs table improve the performance by making insertion and search take $O(1)$ time, but they do add intellectual complexity to the code.

The Warshall and the Floyd-Warshall algorithms are slower than the other algorithms we've discussed so far in this book. They both operate in $O(V^3)$ time, making use of the row and column nature of the adjacency matrix. This is the result of the three nested loops used in their implementation. They can be implemented using adjacency lists to save some time and get to $O(V^2 \times E)$, but at some cost of increased code complexity.

Intractable Problems

In this book you've seen Big O values starting from $O(1)$, through $O(N)$, $O(N \times \log N)$, $O(N^2)$, up to (for the Warshall and the Floyd-Warshall algorithms) $O(N^3)$. Even $O(N^3)$ running times can be reasonable for values of N in the thousands. Algorithms with these Big O values can be used to find solutions to most practical problems.

Some algorithms, however, have Big O running times that are so large that they can be used only for relatively small values of N. Many real-world problems that require such algorithms simply cannot be solved in a reasonable length of time. Such problems are said to be **intractable**. (Another term used for such problems is **NP complete**, where NP means nondeterministic polynomial. An explanation of what this means is beyond the scope of this book.) The very complexity of these problems is part of what makes them interesting. Let's look at a couple of them.

The Knight's Tour

The Knight's Tour (Programming Project 14.6 in [Chapter 14](#)) is an example of an intractable problem because the number of possible paths is so large. The total number of possible move sequences (tours) is difficult to calculate, but you can approximate it. A knight in the middle of a board can move to a maximum of eight squares. This number is reduced by moves that would be off the edge of the board and moves that would end on a square that was already visited. In the early stages of a tour, there will be closer to eight moves, but this number will gradually decrease as the board fills up.

Let's assume (conservatively) only two possible moves from each position averaged over the entire puzzle where it will vary quite a bit. After the initial square, the knight can visit 63 more squares. Thus, there is a total of 2^{63} possible move sequences. This is about 10^{19} . Assume a computer can make a billion moves a second (10^9). There are roughly 10^7 seconds in a year, so the computer can try about 10^{16} moves in a year. Solving the puzzle by brute force—exploring every possible move sequence—can therefore be expected to take 10^3 , more than a thousand years because this was a conservative estimate.

This particular problem can be made more tractable if strategies are used to “prune” the tree created by the move sequences. One is Warnsdorff's heuristic (H. C. von Warnsdorff, 1823), which specifies that you always move to the square that has the fewest possible exit moves. A **heuristic** (from the ancient Greek: εὕρισκω, *heurískō*, “I find, discover”) in computer science, is a technique that helps solve a problem more quickly, although it is not always guaranteed to do so.

The Traveling Salesperson Problem

Here's another famous intractable problem. Suppose you're a salesperson, and you need to go to all the cities where you have potential clients. You would like to minimize the number of miles, and thus time, you travel. You know the distance from each city to every other city. You want to start in your home city, visit each client city only once, and return to your home city. In what sequence should you visit these cities to minimize the total miles traveled? In graph theory, this is called the **traveling salesperson problem**, often abbreviated TSP. (It was originally called the traveling salesman problem.)

Let's return to Turala for another business opportunity, selling networking equipment in each of the six major cities. [Figure 15-21](#) shows the cities and their driving distances. What's the shortest way to travel from Blum through each of the other cities and back to Blum? As before, not all cities are connected by roads to all other cities, at least not without passing through another city. For example, there are many ways to get from Blum to Gray, but none of them include driving directly from Blum to Gray without going through Cerf or Dahl.



Figure 15-21 *Driving distances in Turala*

To find the shortest salesperson route, you could list all the possible permutations of the other cities (Dahl–Kay–Naur–Gray–Cerf, Dahl–Gray–Naur–Kay–Cerf, Dahl–Gray–Kay–Naur–Cerf, and so on) and calculate the total distance for each permutation. The route Blum–Dahl–Kay–Naur–Gray–Cerf–Blum has a total length of 140.

Unfortunately, the number of permutations can be very large: it's the factorial of the number of cities (not counting your home city). If there were 6 cities to visit, there are 6 choices for the first city, 5 for the second, 4 for the third, and

so on, for a total of $6 \times 5 \times 4 \times 3 \times 2 \times 1$, or 720 possible routes. For the moment, you can count all possible permutations, even if no roads connect some pairs of cities.

The problem quickly becomes impractical to solve for even 50 cities (more than 10^{64}). Again, there are strategies to reduce the number of sequences that must be checked, but this helps only a little. You can apply the same logic that was used in the Knight's tour, noting that at each city you choose among a few edges, excluding those going to previously visited cities. If the average number of choices at each city is 2 or more, then there are at least 2^{50} paths to explore when trying to visit 50 cities. That's over 10^{15} , which is much smaller than 10^{64} , but still a huge number.

A weighted graph is used to implement the TSP problem, with weights representing distance and vertices representing the cities. The graph can be undirected if the distance is the same going from A to B as from B to A, as it usually is when driving. If the weights represent airfares or travel times, they may be different in different directions, in which case a directed graph is used.

Hamiltonian Paths and Cycles

A problem that's similar to the TSP but more abstract is that of finding a Hamiltonian cycle of a graph. As we noted earlier, a cycle is a path that starts and ends on the same vertex. A **Hamiltonian path** is one that visits every vertex in the graph exactly once but does not have a final edge that returns to the initial vertex. It's named for William Rowan Hamilton, who invented the *icosian game*, now also known as *Hamilton's puzzle*, which involves finding a Hamiltonian cycle in the edge graph of the dodecahedron. Finding a Hamiltonian path is somewhat like the bridges of Königsberg problem discussed in [Chapter 14](#). The difference is between finding a path that visits all the vertices once, while the bridges of Königsberg problem seeks a path that visits all the edges once (and allows revisiting vertices).

Don't confuse finding a Hamiltonian cycle with detecting whether there are any cycles in a graph. A **Hamiltonian cycle** must include all the vertices, not just a subset, and be a cycle. Cycles in a graph can be detected with Warshall's algorithm (or Floyd-Warshall or some other more targeted algorithms) and detecting their existence is considerably easier than finding a Hamiltonian cycle.

Every solution to the TSP is a Hamiltonian cycle, but unlike the TSP, finding Hamiltonian paths and cycles doesn't care about distances; all you want to know is whether such a cycle exists. In [Figure 15-21](#) the route Blum–Dahl–Kay–Gray–Naur–Cerf–Blum is a Hamiltonian cycle, while Blum–Dahl–Kay–Cerf–Gray–Naur–Blum is not because there is no Naur–Blum edge. The Knight's Tour problem is an example of a Hamiltonian cycle, if the knight returns to its starting square, and a Hamiltonian path, if not.

Finding a Hamiltonian cycle takes the same $O(N!)$ time as the TSP. You'll see the term **exponential time** used for Big O values such as 2^N and $N!$ The term *exponential* is used loosely in everyday parlance to mean anything that increases really fast. For computer science, however, we like to be a bit more precise.

To see how much worse $O(N!)$ and $O(2^N)$ time are than $O(N^2)$, you can plot their growth on a graph (well, a different kind of graph...). [Figure 15-22](#) puts them in perspective with the other growth rates that you saw in [Chapter 2](#), “[Arrays](#).”



Figure 15-22 *Exponential growth*

The uppermost line shows the $O(N!)$ growth rate. It starts off being less than $O(N^2)$ for small values of N but quickly dominates all the others. The next line below it is for $O(2^N)$, and it also dominates all of the other growth rates you've seen so far. It's clear that the number of steps grows far faster for these exponential categories than the others.

The $O(2^N)$ line appears straight in this graph, whereas all the others have some curvature. Is it clear to you why it is straight? In the graph of [Figure 15-22](#), the

vertical axis is *logarithmic*. Every step vertically in the graph means the number grows tenfold. You saw that logarithms are the inverse of exponential functions back in [Chapter 2](#). When you plot an exponential function like 2^N on a logarithmic scale, they “cancel each other,” and the curve flattens out into a line.

The growth rates $O(1)$, $O(\log N)$, $O(N)$, $O(N \times \log N)$, and $O(N^2)$ are all less than exponential, curving downward as N increases in the logarithmic graph of [Figure 15-22](#). Note that while the $O(N^2)$ curve has an exponent in the formula, it is *not* called exponential because it grows significantly more slowly than the $O(2^N)$ curve. Even the more complex $O(N^3)$, $O(N^4)$, and so on growth rates will be smaller than $O(2^N)$ for large values of N .

The $O(N!)$ growth rate not only exceeds the $O(2^N)$ exponential, but it has a slight upward curve. That’s on the logarithmic scale, so it grows even more aggressively than the others do. That’s why you must be very careful with algorithms that have this kind of performance. If you invent a new algorithm to solve a problem, you should analyze its performance to see what size inputs it can handle.

Would you ever want to implement a program that takes exponential time? Probably not, but there’s another reason why they are so intriguing. Certain kinds of problems can be shown to *always* require exponential time to solve. It’s not a question of choosing the right data structures to get a lower complexity solution; there are none. These kinds of algorithms are the basis of cryptography—scrambling information into sequences of symbols that should be accessible only to specific people or groups. Finding really hard problems to solve means the problem’s solution can be used to encrypt the information. Only those people who know the solution will be able to decrypt the information in any kind of reasonable time.

Summary

- In a weighted graph, edges have an associated number called the weight, which might represent distances, costs, times, or other quantities.
- The minimum spanning tree in a weighted graph minimizes the weights of the edges necessary to connect all the vertices.

- An algorithm using a priority queue or heap can be used to find the minimum spanning tree of a weighted graph.
- At each step, the minimum spanning tree algorithm chooses the lowest weighted edge from a visited vertex to an unvisited one.
- Finding the minimum spanning tree of a weighted graph solves real-world challenges such as installing network cables between cities.
- Large graphs are often “discovered” as an algorithm focuses on one vertex, edge, or path at a time.
- Nonadjacent vertices can be used by assigning them an infinite weight edge.
- The shortest-path problem in a nonweighted graph involves finding the minimum number of edges between two vertices.
- Solving the shortest-path problem for weighted graphs yields the path with the minimum total edge weight.
- The shortest-path problem for weighted graphs can be solved with Dijkstra’s algorithm.
- In each iteration of Dijkstra’s algorithm, the lowest weight route to an unvisited vertex determines the next vertex to visit.
- Dijkstra’s algorithm keeps records for the shortest path to all visited vertices including the total weight and parent vertex on the path to that visited vertex.
- The algorithms for large, sparse graphs generally run much faster if the adjacency-list representation of the graph is used rather than the adjacency matrix.
- Hash tables are a good representation for sparse adjacency matrices.
- Finding the total weight of the edges between every pair of vertices in a graph is called the all-pairs shortest-path problem. The Floyd-Warshall algorithm can be used to solve this problem.
- Some graph algorithms take exponential time and are therefore not practical for graphs with more than a few vertices.

- The exponential growth rates— $O(N!)$ and $O(2^N)$ —grow significantly faster than the $O(N^2)$ rate, which in turn, is much higher than other rates like $O(N \times \log N)$, $O(N)$, and $O(\log N)$.
- Examples of problems that take exponential time are the traveling salesperson problem (TSP) and finding Hamiltonian cycles.

Questions

These questions are intended as a self-test for readers. Answers may be found in [Appendix C](#).

1. The weights in a weighted graph are used to model things like _____, _____, and _____.
2. The minimum spanning tree (MST) of a weighted graph minimizes
 - a. the number of edges from the starting vertex to a specified vertex.
 - b. the number of edges used to span all the vertices.
 - c. the total weight of edges connecting all the vertices.
 - d. the total weight of the edges between two specified vertices in the tree.
3. The numerical weights in weighted graphs
 - a. must be integers with zeros reserved for nonadjacent vertices.
 - b. can be any finite value but must have a sum less than the number of vertices.
 - c. cannot include negative values.
 - d. can include any value including positive infinity for nonadjacent vertices.
4. True or False: The weight of the MST depends on the starting vertex.
5. In the MST algorithm, what is removed from the priority queue?
6. In the country network installation example, at the time each edge is added to the MST, the edge connects
 - a. the starting city to an adjacent city.
 - b. an already-connected city to an unconnected city.

- c. an unconnected city to a fringe city.
 - d. two cities with offices.
7. True or False: After adding an edge to the MST, it could be removed or replaced later with a better edge.
 8. When exploring edges at a newly visited vertex, the MST algorithm “prunes” edges that lead to a vertex that _____.
 9. True or False: The shortest-path problem must be carried out on a directed graph.
 10. Dijkstra’s algorithm finds the shortest
 - a. paths from one specified vertex to all the vertices visited while finding the shortest path to another specified vertex.
 - b. paths from all vertices to all other vertices that can be reached along one edge.
 - c. paths from all vertices to all other vertices that can be reached along multiple edges.
 - d. path from one specified vertex to another specified vertex.
 11. True or False: The rule in Dijkstra’s algorithm when applied to a graph where the edge weights are distances is to always put in the subgraph the unvisited vertex that is closest to the starting vertex.
 12. In the railroad route example, a fringe city is one
 - a. to which the travel time is known, but from which no other travel times are known.
 - b. which is in the tree.
 - c. to which the travel time is known and which was just added to the tree.
 - d. which is completely unknown.
 13. The all-pairs shortest-path problem involves finding the shortest path
 - a. from the starting vertex to every other vertex.
 - b. from the starting vertex to every vertex that is one edge away.
 - c. from every vertex to every other vertex that is more than one edge away.

- d. from every vertex to every other vertex.
14. Comparing the Floyd-Warshall algorithm with Warshall's algorithm (described in [Chapter 14](#)), how are the matrix cells updated differently?
 15. Problems that take an exponential amount of time to solve are called _____.
 16. Representing adjacency using a(n) _____ can reduce the complexity of several algorithms on sparse graphs compared to using a(n) _____.
 17. A path weight matrix is the output of the _____ algorithm.
 18. What is an approximate Big O time for an attempt to solve the knight's tour on a $K \times K$ board?
 19. In [Figure 15-21](#), is the route Blum–Cerf–Naur–Kay–Gray–Dahl–Blum the minimum solution for the traveling salesperson problem? Why or why not?
 20. When should a directed graph be used to solve the traveling salesperson problem, as opposed to a undirected graph?

Experiments

Carrying out these experiments will help to provide insights into the topics covered in the chapter. No programming is involved.

15-A Consider an alternative proposal for finding the minimum spanning tree. For a connected, undirected graph with V vertices, take the $V - 1$ lowest weight edges and make a graph with them and the V vertices. If some of the edges have the same weight, randomly select among the edges of equal weight to get the $V - 1$ edges. Try it on the graphs of [Figure 15-1](#) and [Figure 15-8](#). Does it provide a better solution? Why or why not? Can you think of other graph examples where it would work or not work?

15-B The minimum spanning tree algorithm can start from any vertex. If it starts at different vertices, will it find the same tree? Can you find some small graphs that produce different trees and some graphs that produce the same trees? (Hint: Keep the graphs small.) See whether you can define under what graph conditions the algorithm will find the exact same minimum spanning tree.

15-C Dijkstra's algorithm finds the shortest path between two vertices and has the side benefit of finding the shortest path from the starting vertex to some of the other vertices. Can you define which of those other vertices get that benefit? Does it matter what ending vertex was chosen? Try some examples with graphs in the WeightedGraph Visualization tool, where the graph has only one connected component.

15-D Traveling salespeople are much rarer now than they used to be. Think about what other kinds of business activity need to solve the TSP. How fast do they need to solve it? How many vertices would be in the graphs?

15-E Turala's government tells you they want to connect water supplies between their different regions using pipelines. They would like to pump water in either direction between about 50 water sources depending on changing rainfall and other conditions. Of course, they want to minimize the costs, and it's acceptable to pump water via intermediate points rather than connecting all sources to all other sources. What kind of problem is this, and what algorithm would you use? What's the difference between finding the minimum construction costs versus minimum operational costs?

Programming Projects

Writing programs to solve the Programming Projects helps to solidify your understanding of the material and demonstrates how the chapter's concepts are applied. (As noted in the Introduction, qualified instructors may obtain completed solutions to the Programming Projects on the publisher's website.)

- 15.1 Create an `allShortestPathsMatrix()` method for the `WeightedGraph` class using the Floyd-Warshall algorithm. It should return an array of the minimum costs to get from any vertex to any other vertex. The result matrix can be a Python `dict` that is indexed like the `WeightedGraph._adjMat` attribute. Demonstrate it running on the train times shown in the lower graph of [Figure 15-13](#).
- 15.2 Create a *directed* weighted graph class based on the `WeightedGraph` described in the text. Use it to make a generator for all maximum length paths in the directed graph. A maximum length path from starting vertex `K` is one that cannot be extended to an adjacent vertex without revisiting a vertex already on the path. The `allMaxPaths()` generator should yield

all such paths starting from all vertices. This can be done as a recursive generator. The generator will produce all the long paths but will also include shorter and even single vertex paths for vertices with no outgoing edges. Create another generator, `HamiltonianPaths()`, that calls `allMaxPaths()` and yields only those paths that contain all the vertices in the graph. Show the output of both methods operating on three graphs: the one shown in [Figure 15-19](#), that same graph minus the outgoing edges from Vertex B, and that same graph with an additional edge from C to B with weight 15.

15.3 Implement a method that solves the traveling salesperson problem described in the “[Intractable Problems](#)” section in this chapter. In spite of its intractability, your method should have no trouble solving the problem for small N, say 10 cities or fewer. Use a directed graph as implemented in Programming Project 15.2. You can use the brute-force approach of testing every possible sequence of cities. Your method should return either an empty list or `None` if there is no solution. Demonstrate your method running on the driving times shown in [Figure 15-21](#) with three variations on the return times: (a) the same time, (b) the return takes 5 minutes longer, and (c) the return takes 10 minutes longer. Does the solution change among the variations?

15.4 When you’re planning a project, it’s important to know how long it will take to complete. You saw in [Chapter 14](#) that directed graphs could be used to model dependency relationships and how to topologically sort the vertices to show the order of tasks needed to complete a project. With time as the weight in a graph, you can find two things: the total time needed to complete the project and the **critical path**—the path through the dependency graph that takes the longest to complete.

Use the directed graph as implemented in Programming Project 15.2 and create a `criticalPath()` method that takes a vertex index as parameter and returns the critical path leading to it along with the total time (weight) along that path. Vertices with no predecessors (inbound edges) have only themselves in the critical path and a total time of zero. All other vertices return the critical path among their predecessors that has the longest time. This method can be implemented recursively. Take care to prevent infinite recursion if the graph has a cycle; the `criticalPath()` method should return the path up to a vertex that would cause a cycle along with infinite time for that path. Show the output of the method operating on three graphs: the one shown in

Figure 15-19, that same graph minus the outgoing edges from Vertex B, and that same graph with an additional edge from B to C with weight 35.

- 15.5 Many problems require finding the connected components of a graph and uniquely labeling the vertices that comprise them. For a road network in a mountainous, snowy country, the towns and cities are all part of one connected component in good weather. As snow falls in the mountains, roads become impassable, sometimes cutting off towns from the others. Weighted graphs can model that situation by having towns as vertices, roads as edges, and weighting the edges by their maximum elevation. When snow cuts off travel above a given elevation, the connected components change.

Write a `connectedComponents()` method for the `WeightedGraph` class that returns an array of labels for each of the vertices. Vertices with the same label in the array are part of the same component; different labels imply disconnected components. The `connectedComponents()` method takes a threshold argument that will be used to identify which roads are still passable. Its default value is infinity so that all roads (edges) with finite weight are included.

The algorithm for finding the labels starts off by assigning each vertex the label of its `name` attribute (you can assume all the cities have unique names). Then you make a sequence of update passes to change the labels. In each update pass, the algorithm looks at every edge with weight below the threshold. If the vertices at the ends of the edge have different labels, it replaces the higher label with the lower one (by comparing their lexicographic order). This repeats until an update pass goes through all the edges without changing any labels. The minimum labels “spread” across all the vertices in their connected component until no more labels change.

Write a second method, `componentVertices()`, that takes the array of labels and builds a hash table that maps a label to a list of vertex indices that share the label. The number of keys in the hash table is the number of connected components. Demonstrate your methods running on the graph in Figure 15-21 using thresholds of 50, 21, and 15.

16. What to Use and Why

In This Chapter

- [Analyzing the Problem](#)
- [Foundational Data Structures](#)
- [Special-Ordering Data Structures](#)
- [Sorting](#)
- [Specialty Data Structures](#)
- [External Storage](#)
- [Onward](#)

We briefly summarize what we've learned so far, with an eye toward deciding what data structure or algorithm to use in a particular situation.

This chapter comes with some caveats. Of necessity, it's very general. Every real-world situation is unique, so what we say here may not be the right answer to your problem. You need to analyze the problem to determine its characteristics and see whether they match those of the data structure or algorithm.

This chapter is divided into these somewhat arbitrary sections:

- **Analyzing the problem:** Determine the characteristics that guide the choice of data structure and algorithm
- **Foundational data structures:** Arrays, linked lists, trees, hash tables
- **Special-ordering data structures:** Stacks, queues, priority queues, heaps
- **Sorting:** Insertion sort, Shellsort, quicksort, mergesort, heapsort

- Specialty data structures: Quadtrees, graphs
- External storage: Sequential storage, indexed files, B-trees, hashing

Note

For detailed information on these topics, refer to the individual chapters in this book.

Analyzing the Problem

As a developer, you should always review the plans for the software you intend to develop. There can be—and usually are—many goals for a particular software project. Commercial companies want to create products that sell. Scientific organizations want systems that accurately record, model, and analyze their data. Governments want software that helps educate, protect, and account for their citizens and jurisdictions. Analyzing these kinds of goals is important; it tells you what needs to be built or changed. The organization's goals are less informative about *how* to build it to achieve the speed, cost (in terms of memory and other computing resources), and ease of maintenance that everyone wants.

Let's assume someone has analyzed the overall goals and provided a set of desired features and requirements for the new system. To figure out what data structures and algorithms are appropriate, you need to look at the characteristics of the computations involved. What kind of data is being manipulated? How much will be processed? What kinds of results are needed and how fast? The answers to these kinds of questions help narrow your choices.

What Kind of Data?

The problem domain usually makes very clear what type of data will be manipulated. It could be records about company transactions, audio recordings of animal calls, measurements of experiments in a lab, or videos that will be streamed to viewers. Each kind of data is built up from the primitive data types: numbers, characters, and Booleans. The primitives are grouped into meaningful chunks—for example, text message strings, arrays of numbers measuring temperatures, records for personnel, centerlines of roadways, medical images,

and volumetric shapes. The enormous variety of kinds of data can be quite daunting.

Despite that variety, some characteristics help pick the best approach for data structures. First among them is how will the different parts of the data be accessed? For example, if you're processing text message strings, you expect them to be short and to be requested as a whole; there likely is no need to extract the last word from each text nor to find the median-valued character in each one. Typically, the "chunks" are manipulated as a whole. If components within the chunks need separate processing, that fact should be noted. For example, videos typically contain a series of still images and a separate series of numeric values for one or more audio channels. The visual and audio content can be manipulated together—as an editor might carve up a video of an interview—or manipulated separately to find utterances of words or find the appearances of blue sky in the video frames.

With the different chunks of data identified, the next question to ask is how will those chunks be identified? In particular, is there a key or index that helps uniquely identify it? For example, personnel records usually have many fields, and several of them might be used as keys to identify the records. Text messages don't have an implicit key to identify them, but when they are part of records, they often have keys for sender, recipient, and time sent. The text content can, of course, be processed to find other keys like words, phrases, or languages used. Keys are often stored alongside the more numeric structures like arrays or images. For example, an image database might have keywords associated with the image content—for example, outdoor/indoor, people, landscapes, animals.

The other way to identify chunks of data is by index or coordinates. To get the next message in a sequence means going to the next index. To get the image that appears 10 seconds after the start of a video, a program would find the video frame by its index. The array-like data chunks always have natural indices and are frequently accessed by those indices. It would make no sense to store an image, for example, with each of its pixels in a different node of binary search tree ordered by pixel value. Reconstructing the array form of the image from the tree would be time-consuming and likely to be needed frequently. Other indices can involve multiple numbers like map coordinates and volume/spatial coordinates.

How Much Data?

When you know the kinds of data, how they fit into chunks, and how they are going to be addressed, the next question is how many of those chunks are there? This characteristic often drives the choice of data structure significantly and is sometimes the hardest to determine. Projects often start with a small scope in mind and only later does someone try to run it with significant amounts of data. It's easy and tempting to focus on the smaller sizes, because that means the program is likely to run fast enough regardless of the data structure(s) and algorithm(s) used. With a little more analysis, you can often foresee how the amount of data might expand and be prepared for it.

In the analysis, you don't have to be very precise. You really only need to know how many data items in terms of **orders of magnitude**. Are there tens of items? Hundreds? Thousands? The exact number doesn't matter, but the number of decimal places does.

Look for External Constraints

If the data to be processed is constrained in some way, you may be able to conclude that the code will never need to process more than a certain amount. If you know, for example, that the system will need to store the known contacts of only one person, it's extremely unlikely to need to store more than 1,000 items and certainly fewer than 100,000. A system tracking one or two items per person for everyone alive today needs to handle at most tens of billions of items. By the time such a system will need to handle trillions, it's likely that some new system will have replaced it.

What Creates the Data?

The biggest difference in amounts of data comes from how it is produced. If the data is manually produced, it is likely to be far smaller than data produced by automation. Things that people produce manually like text messages, songs, or paintings are all limited by the amount of time spent doing the activity and the number of people doing it. There are good models about how many people are alive now and how many there will be for decades. It may be less clear how many of the people will perform the activity, but you really only need estimates of the order of magnitude: thousands, millions, and so on.

Data produced by automation can be many orders of magnitude larger. Imagine all of the video produced by an astronomical telescope operating continuously (or at least nightly) for a year. Multiply that by all the telescopes in operation

and the total time period to get an estimate for the quantity of data that might be collected for analysis. When making estimates, you must also consider potential growth. For example, when you're thinking about the number of sources of video that might produce content on a video sharing site, an estimate made in the year 2000 would probably have been one or two orders of magnitude smaller than one made in 2020 after the widespread acceptance of handheld smartphones with video recording capability.

Underestimates are common. Projects usually start by focusing on small goals and leave the challenge of scaling for later. A classic example was the common design decision to store two digits for the year in many database systems developed in the late twentieth century. Companies and governments made great efforts to find and fix all such limited systems before the year 2000. Similar problems will occur in the future too, because nearly all timekeeping systems use a fixed amount of storage for their timestamps. Although these examples are not underestimates of the quantity of data, they are indicative of the tendency to develop systems that can handle initial estimates without accounting for how the data will change over time.

What Operations and How Frequent?

The system to be built does something with these chunks of data. A good design lists all the operations, along with estimates of how frequently they will be done and what the requirements are for their performance.

For example, a bookkeeping system for a business has insertion of new data for every sales transaction, income, and payments. There are hundreds or thousands or (hopefully) hundreds of thousands of transactions daily. There are deletions for errors occasionally, maybe a hundred in a day. There are operations to report on overall activity, income and expenses, every day, week, month, quarter, and year. There are performance requirements for each of those, such as being able to insert a new transaction in a microsecond or get a quarterly report in less than a minute. All of those can be reasonably achieved, but on some new project, you might spot problems where it will become difficult to do.

It's often quite clear what kinds of operations must be done for inputs and outputs to humans. Operations that are done for maintenance or for sending to some other process might have time constraints that are less clear. For example, storm prediction models need to be rerun with the latest inputs at regular

intervals, legal requirements can require that old data be removed from a system periodically, and financial transactions that streamed in too fast to be fully processed need to be reconciled and cataloged after trading stops. The term *background processing* is sometimes used to describe such maintenance tasks. They can affect all the system's data and hence could take significant computing resources.

When you're considering how frequently operations occur, it's not always possible to be specific. Unlike the bookkeeping system with daily, weekly, monthly, quarterly, and annual reports, most systems are run on demand, and the amount of demand is highly variable. What's most important to know, however, is the *relative frequency* of the operations. Are searches more frequent than insertions? Is traversal more frequent than search? At a minimum, you should try to rank order the four basic operations of insertion, deletion, search, and traversal.

The processing order of data items strongly influences the choice of structures. For example, reconciling all the transactions that streamed in during a trading day likely must be done in the order they occurred. That means a queue is more appropriate than a stack. If the streaming order is not quite the same as the trading order, then a priority queue or heap would be better.

In another example, a warehouse inventory system would have updates for new stock added to the warehouse and for items removed to fill orders. When users want to place an order, the current item quantities determine whether or not it can be filled immediately. The sequence of what items appear in an order is not predictable. The orders contain seemingly random items. This means that the processing order is unpredictable, and using a data structure designed for a specific order like a queue or a stack might be ill advised.

Different algorithms can impose the processing order requirement as well. In Dijkstra's algorithm for the shortest path between two vertices in a graph ([Chapter 15](#), "[Weighted Graphs](#)"), there is a specific order in visiting the vertices. In the Tower of Hanoi problem ([Chapter 6](#), "[Recursion](#)"), both the constraints on moving disks and the method of finding the specific order of disk movements to solve the puzzle need the last-in, first-out (LIFO) processing order provided by a stack. The algorithmic constraints are sometimes better defined than what the system requirements provide.

Most importantly, what operations must be fast? Are any of them special? If the purpose of the system is to produce the driving routes of delivery trucks or to

find the locations of warehouses nearby, the general-purpose data structures are not likely to perform well enough. The operations that must be performed the fastest often force the choice of using specialized data structures and/or algorithms.

Who Will Maintain the Software?

Determining what data structures and algorithms to use depends not only on the data and operations on that data, but on the people who will maintain the software as (the inevitable) changes are made. Programmers have different skill levels, and using highly complicated algorithms or structures could become a barrier to being able to make changes easily.

Knowing who will work on a problem and what kinds of data structures are challenges for them is probably the hardest problem characteristic to gauge. If this is a program run by a large corporate or government agency, they are likely to be able to find highly skilled programmers to maintain and update the system. At the other end of the spectrum, a program written by volunteers for a social club might have trouble finding more volunteers to fix some future problem. Nontechnical organizations could have more trouble than ones that attract volunteers with software skills.

If you expect the people maintaining the software to be comfortable with advanced data structures, then the choices of structures and algorithms are wide open. Technical and scientific organizations are generally very receptive to using complex structures, if they will achieve better performance. Another way to put it is that technical organizations are less averse to the risk of using advanced structures and algorithms that might be hard for software developers to master in later updates. That doesn't mean you should always choose the most advanced structure or algorithm for such an organization; choosing the least complex one that meets the performance requirements means a broader group of developers will be able to maintain it.

When the future development staff is less well-defined, or is unlikely to have familiarity with advanced structures, or would take a long time to learn complex structures, the design choices should be more limited. Using exotic algorithms that are not widely known adds to the maintenance burdens for the system. That said, it is always important to achieve the performance requirements of the system. It would be unacceptable, for example, to use an $O(N^2)$ algorithm that's easier to understand than a complex $O(N \times \log N)$

algorithm if the system is not able to process the expected amount of data in the required time.

Note that some complexity issues are mitigated by the use of standard libraries to implement the data structures. Hash tables use some rather complex concepts in their implementation, but using them inside another program is straightforward when a good library is available. Data structures in standard libraries with well-documented programming interfaces and well-known performance characteristics should be fairly easy for other developers to use and maintain.

Foundational Data Structures

Every data structure is built on top of the primitive types—numbers, characters, and Booleans—and ways of putting them together. The two fundamental ways of arranging these primitives into bigger groups are by putting sequences of the same primitive type into arrays, and by linking different chunks together using references. You can also say that references are a primitive type (although you use them only for organizing the other data, not to hold the values to be manipulated).

With these basic constructs, you can form the foundational data structures: arrays, linked lists, and trees. You can also include hash tables in this category. Hash tables behave like arrays in many ways (even though the indexing is much more complicated than for arrays). All other structures can be built using combinations of these, but sometimes they are all that is needed.

The biggest differences among these foundational structures are

- How items they contain are addressed: by index, by reference, by key, by coordinates
- How memory is allocated and deallocated for them

Arrays, of course, allow items to be indexed by an integer and are allocated in blocks where the program specifies the number of items when the array is created. Linked lists and trees make use of references. The items they contain can be specified by a reference to a link or node, or perhaps by an integer that counts items from the beginning of the list or the root of the tree. The links and nodes are allocated as they are needed. Hash tables address items by a key and typically only by a key. Of course, items in arrays, linked lists, and trees can

also be addressed by a key, but it takes more time to do so, significantly more time as the number of items grows. Hash tables are also somewhat like lists and trees in the way that memory can be allocated as needed.

Which of these general-purpose data structures is appropriate for a given problem? We go through them individually and show how the characteristics of a problem help answer that.

Speed and Algorithms

The general-purpose data structures can be roughly arranged in terms of speed when items are specified by key: arrays and linked lists are slow, trees are fairly fast, and hash tables are very fast.

Don't draw the conclusion, however, that it's always best to use the fastest structures. There's a penalty for using them. First, they are—in varying degrees—more complex to program than the array and linked list. Also, hash tables use memory somewhat inefficiently. Ordinary binary trees revert to slow $O(N)$ operation for ordered data; and balanced trees, which avoid this problem, are more complex to program.

Processing Speed: A Moving Target

The fast structures come with some drawbacks, and another development makes the slow structures more attractive. Every year there's an increase in the CPU and memory-access speed of the latest computers. Moore's Law (postulated by Gordon Moore in 1965) specifies that the number of transistors on microchips will double every 18 months. Over 50 years later, that trend has held up, even as the size of individual transistors approaches the size of molecules and atoms.

In a related trend, the clock speed of chips rose exponentially, although the rate of increase has begun to diminish. Coupled with the ability to run many programs (or threads) in parallel, the effective processing speed of computers has continued to rise exponentially. Doesn't this suggest that you don't need to worry about the big O efficiency of data structures? If the computer runs exponentially faster, won't it overcome the differences between the slower and faster structures?

The answer is not a simple yes or no. While the computers have been able to run faster, software developers have taken advantage of these gains to write

software that uses more memory and processing cycles on the computer. This **software bloat** means that operating system size, random-access memory requirements, library size, and features have all been increasing as well. Ironically, those increases have reduced and occasionally reversed the speed gains from hardware. The instinct to implement “easier-to-use” but slower or more memory-intensive structures has counteracted the gains in processing speed.

Suppose a computer a few years ago handled an array of 100 objects in acceptable time. Now, computers are much faster, so an array with 10,000 objects might run at the same speed. Many writers provide estimates of the maximum size you can make a data structure before it becomes too slow. Don’t trust these estimates (including those in this book). Today’s estimate doesn’t apply to tomorrow.

Instead, start by considering the simplest data structures that meet the requirements. Unless it’s obvious they’ll be too slow, code a simple version of an array or linked list and see what happens. If it runs in acceptable time on the biggest amount of data, look no further. Why toil away on a balanced tree when no one would ever notice if you used an array instead? Even if you must deal with millions or tens of millions of items, it’s still worthwhile to see how well an array or linked list will handle them. Only when experimentation shows their performance to be too slow should you revert to more sophisticated data structures. Of course, you must plan properly to have time to make such changes if your initial design doesn’t pan out.

Libraries

Libraries of data structures are available commercially in all major programming languages. Languages themselves may have some structures built in. We’ve explored Python’s structures extensively throughout this book. Java, for example, includes `Vector`, `Stack`, and `Hashtable` classes. C++ includes the Standard Template Library (STL), which contains classes for many data structures and algorithms.

Using a commercial library may eliminate or at least reduce the programming necessary to create the data structures described in this book. It also eliminates some of the risk that future developers will need to dig into the details of their coding. When commercial or well-maintained public libraries are available, using a complex structure such as a balanced tree, or a delicate algorithm such

as quicksort, becomes a more attractive possibility. You must, however, analyze their specific features, requirements, and limitations to ensure that the class, method, or algorithm can be adapted to your particular problem.

Arrays

In many situations the array is the first kind of structure you should consider when storing and manipulating data. Arrays are useful when

- The amount of data is reasonably small.
- The amount of data is predictable in advance.
- The items will be addressed by index for most of the time.
- The order the items will be accessed is variable.

What is “reasonably small”? The answer depends on your computing resources and how much you must store for each array cell. The basic idea is that it should fit in the computer’s random-access memory (RAM). It can be as large as the virtual address space allowed on the computer, but parts of that space will be swapped in and out of the random-access memory, slowing it down. As an example, a single processor might have 8 gigabytes of RAM and a virtual address space of 2^{64} bytes. Allocating a billion-cell array (where each cell takes 8 bytes) might be fine in that environment. The larger the array, the more the speed of shuffling data in and out of RAM will slow down the computation. The shuffling becomes even more pronounced on systems where multiple processes must share the RAM. We discuss this issue more in the “[Virtual Memory](#)” section.

Predicting the size of the data in advance is hard in many applications. Arrays work best when the whole array can be allocated at once, or perhaps in a few iterations. When the array size grows, data must be copied from the existing array to the new one. The growth operation can sometimes lead to a significant amount of memory being allocated but unused.

The speed of arrays is contingent on being able to find a cell in constant time by using an integer index to find where it lies in memory relative to the array’s starting address. If the algorithms that use the items in the array have that integer index available, arrays are as fast as it gets. If the items must be moved within the array, say within a sorted array or an array where the active data

must always be the first N cells, then the indices change with deletions and/or insertions, and it becomes harder to take advantage of the constant time access. Arrays are best when the data will be inserted, randomly accessed by the index, and rarely deleted.

You shouldn't use an array

- If the data will be addressed by key (try a hash table).
- If there will be many insertions and/or deletions that require moving items within the array.
- If the data must be kept in sorted order and is large (try a heap).

Linked Lists

Consider a linked list when

- The amount of data to be stored is not easily predicted in advance and minimizing memory usage is important.
- Data will frequently be inserted and deleted, and the order of the items is important.
- The data will be addressed or processed in the order it was received (or the reverse).

The linked list obtains whatever storage it needs as new items are added, so it can expand to fill all available memory; and there is no need to fill “holes” during deletion, as there is in arrays.

Insertion is fast in an unordered list. Searching and deletion by key or by index are slow (although deletion is faster than in an array). Like arrays, linked lists addressed by key or index are best used when the amount of data is comparatively small.

You shouldn't use a linked list

- If the data will be addressed by a key (try a hash table).
- If the data will be addressed by an integer index in arbitrary order (try an array).

Binary Search Trees

A binary search tree is the next structure to consider when arrays and linked lists prove too slow. Consider binary search trees

- When data will be addressed and ordered by a key.
- When data will be frequently inserted and deleted.
- When minimizing memory usage is important.

A tree provides fast $O(\log N)$ insertion, searching, and deletion. Traversal in key order is $O(N)$, which is the fastest for any data structure. [The fact that binary search trees order their items by a key means a traversal in sorted order takes $O(N)$ time while an unsorted array or list would need to be sorted first.] You can also find the minimum and maximum quickly, $O(\log N)$, and traverse a range of items.

An unbalanced binary tree is much easier to program than one that balances itself, but unfortunately ordered data can reduce its performance to $O(N)$ time, no better than a linked list. If you're certain the data will be inserted in random order with respect to the key, there's not much point in using a balanced tree.

You shouldn't use a binary search tree

- If an array or a linked list meets all the requirements.
- If the order of the items is unimportant.
- If the items are likely to be inserted in the forward or reverse order of the key.

Balanced Search Trees

Of the various kinds of balanced search trees, we discussed 2-3-4, red-black, and AVL trees. They are all balanced trees, and thus guarantee $O(\log N)$ performance whether the data is inserted ordered according to the key or not. These balanced trees are challenging to program, with the red-black tree being the most difficult. They also impose slightly more memory overhead than plain binary search trees, which may or may not be significant.

Consider a balanced search tree

- When data will be addressed and ordered by a key.
- When long sequences of insertions in (reverse) order of the key are likely.
- When data will be frequently inserted and deleted.
- When minimizing memory usage is important.
- When the complexities of the balancing algorithms are handled by experienced developers.

The risk associated with developing and maintaining complex algorithms may be reduced by using libraries maintained by qualified companies or other organizations. In many cases hash tables may be a better choice than balanced search trees, but also may use complex hashing algorithms.

You shouldn't use a balanced search tree

- If an array or a linked list meets all the requirements.
- If the order of the items is unimportant.
- If the items are likely to be inserted in random order with respect to the key.

Hash Tables

Hash tables have the most desirable performance for almost all data items referenced by a key. The keys can be any data type that might be considered an index, including integers. The $O(1)$ performance for searching, insertion, and deletion by key are the fastest possible.

Consider a hash table

- When data will be addressed by a key and the traversal order of the items by the key is unimportant.
- When minimizing memory usage is somewhat important and some unused memory is tolerable.
- When the complexities of the hashing and collision resolution algorithms are handled by experienced developers.

Hash tables are not sensitive to the order in which data is inserted and so can take the place of a balanced tree. Programming them is simpler than for balanced trees assuming a good hashing function for the keys is available.

Hash tables require more than the minimum amount of memory, especially for open addressing. For example, with a load factor limit of 50 percent, they use double the number of cells or more than the links needed for a linked list. Like the linked lists and trees, the memory can be allocated as items are inserted with only a modest amount of extra time. (Some insertions cause the array to grow, but over N insertions, only time proportional to N is needed to copy items.)

Hash tables don't support any kind of ordered traversal, or easy access to the minimum or maximum keys and items. Even finding all the keys that have been inserted is more time-consuming than a tree or list. If these capabilities are important, the binary search tree is a better choice.

You shouldn't use a hash table

- If the order of the items by key is important (try a balanced search tree).
- The amount of memory used must be as small as possible (try a balanced search tree).
- The keys of items are not hashed uniformly by the hashing function (try another hashing function).

Comparing the General-Purpose Storage Structures

[Table 16-1](#) summarizes the speeds of the various foundational data storage structures using Big O notation.

Table 16-1 *Foundational Data Structure Speeds*

Data Structure	Search	Insertion	Deletion	Traversal
Array	$O(N)$	$O(1)$	$O(N)$	$O(N)^*$
Ordered array	$O(\log N)$	$O(N)$	$O(N)$	$O(N)$
Linked list	$O(N)$	$O(1)$	$O(N)$	$O(N)^*$
Ordered linked list	$O(N)$	$O(N)$	$O(N)$	$O(N)$
Binary tree (average)	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(N)$
Binary tree (worst case)	$O(N)$	$O(N)$	$O(N)$	$O(N)$
Balanced tree (average and worst case)	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(N)$
Hash table	$O(1)$	$O(1)$	$O(1)$	$O(N)^*$ (but higher C)

In unordered arrays and lists, insertion is made at the end and beginning of the structure, respectively. Deletions within arrays requires filling the gap by shifting items. Deletion of the first item of a list is $O(1)$, but the general delete by key takes $O(N)$. The ordered array uses a binary search, which is fast, but insertion and deletion require moving half the items on the average, which is slow. Traversal means visiting all N items, but only the ordered structures make it easy to visit them in the order of ascending (or descending) keys; the asterisk (*) means the traversal by ordered keys is not supported.

Special-Ordering Data Structures

The special-ordering data structures discussed in this book are the stack, the queue, and the priority queue. These structures provide fast access to the data in a particular order. Instead of acting as a general-purpose data store, they are typically used by a computer program to aid in carrying out some algorithm. We've seen examples of this kind of usage throughout this book, such as in [Chapters 14](#), "[Graphs](#)," and [15](#), "[Weighted Graphs](#)," where stacks, queues, priority queues, and hash tables are all used in graph algorithms.

Stacks, queues, and priority queues are implemented by a foundational structure such as an array, a linked list, or (in the case of the priority queue) a

heap. These data structures present a simple interface to the user, typically allowing only insertion and the ability to access or delete only one particular data item. These accessible items are

- For stacks: The last item inserted
- For queues: The first item inserted
- For priority queues: The item with the highest priority (and the first item among items with the same priority)

These abstract data types (ADTs) can be seen as conceptual aids. Their functionality could be obtained using the underlying structure (such as an array) directly, but the well-defined interface makes it clear to any programmer that their purpose is to provide fast access in a specific order. For example, imagine two similar programs, one with an array (`list` in Python) and one with a queue object. They might perform the exact same operations on the underlying cells, but the very presence of the queue signals that the data will be handled in first-in, first-out (FIFO) order.

The main drawback of these ADTs is that they can't be conveniently searched for an item by key value. They can be traversed easily, but only the priority queue can be quickly traversed in key order (if it is not implemented as a heap).

Stack

A stack is used when you want access only to the last data item inserted; the other items are accessed only after doing something with last data item. It's a last-in, first-out (LIFO) structure. It also allows fast peeking at the first item inserted on the stack but not deleting it.

A stack is often implemented as an array or a linked list. The array implementation is efficient because the most recently inserted item is placed at the end of the array, where it's also easy to delete. Stack overflow can occur but is not likely if the array is adequately sized.

If the stack will contain a lot of data and the amount can't be predicted accurately in advance (as when recursion is implemented as a stack), a linked list is a better choice than an array. A linked list is efficient because items can be inserted and deleted quickly from the head of the list. Stack overflow doesn't occur until the entire memory is full.